Funktioner - Svar

Bemærk: Kodeløsningerne ligger i mappen `files/solution/`. Denne fil indeholder forklaringer, PBL-svar og henvisninger til løsningsfilerne.

Svar: Funktioner 1 - Kodens læsbarhed

Løsningsfil: `files/solution/functions1_with_comments.c` (og `functions1_with_functions.c`)

Svar: Det gør koden mere læsbar uden at ændre funktionaliteten:

- **Meningsfulde variabelnavne:** Erstat kryptiske navne som `x`, `y` og `a` med beskrivende navne som `values_above_element`, `element_count` osv.
- **Kommentarer:** Tilføj kommentarer, der forklarer logik, som ikke er indlysende, komplekse algoritmer eller forretningsregler
- **Ensartet indrykning:** Brug den samme indrykning og afstand i hele koden
- **Fornuftig brug af mellemrum:** Sæt mellemrum omkring operatorer og efter kommaer

PBL-svar: Meningsfulde variabelnavne er vigtige, fordi:

1. **Selvdokumenterende kode:** Variabler med gode navne forklarer, hvad de repræsenterer, uden at der er brug for kommentarer
2. **Vedligeholdelse:** Når du eller andre vender tilbage til koden senere, er den meget lettere at forstå
3. **Debugging:** Klare navne gør det lettere at se, hvad hver variabel gør, når man leder efter fejl
4. **Samarbejde:** Holdkammerater kan hurtigere forstå din kode
5. **Færre fejl:** Misvisende navne kan føre til misforståelser og dermed fejl

Eksempel: `N(211)=15` bliver meget tydeligere, når man ved, at `N` betyder "antal", og parametrene har meningsfulde navne.

Svar: Funktioner 2 - Funktion til BMI-beregning

Løsningsfil: `files/solution/functions2_sol.c`

Forventet output:

```
BMI=18.12
```

(For weight = 31.1, height = 1.31. Løsningsfilen bruger height = 1.8 og weight = 63.0 og udskriver derfor `BMI=19.44`.)

PBL-svar: Ændret funktion, der også udskriver BMI-kategorien:

```c
#include <stdio.h>

float calculateBMI(float height, float weight) {
    float bmi = weight / (height * height);

    // Udskriv kategori
    if (bmi < 18.5) {
        printf("Category: Underweight\n");
    } else if (bmi >= 18.5 && bmi < 25.0) {
        printf("Category: Normal weight\n");
    } else if (bmi >= 25.0 && bmi < 30.0) {
        printf("Category: Overweight\n");
    } else {
        printf("Category: Obese\n");
    }

    return bmi;
}

int main() {
    float weight = 31.1;
    float height = 1.31;
    float bmi = calculateBMI(height, weight);
    printf("BMI=%.2f\n", bmi);
    return 0;
}
```

Svar: Funktioner 3 - Udskriv værdi med pointer

Løsningsfil: `files/solution/functions3_sol.c`

Svar:

- Hvilken information sendes til `printValue`? **Hukommelsesadressen** på variablen `value` (en pointer til en int)

Forventet output:

```
address: 0x7ffd42a1b3ac, value: 101
```

(Adressen er forskellig fra kørsel til kørsel)

PBL-svar: Ændret, så pointerens størrelse også udskrives:

```c
#include <stdio.h>

void printValue(int *pval) {
    printf("address: %p, value: %d\n", (void *)pval, *pval);
    printf("Size of pointer: %zu bytes\n", sizeof(pval));
}

int main() {
    int value = 101;
    printValue(&value);
    return 0;
}
```

Outputtet viser pointerens størrelse (typisk 4 bytes på 32-bit systemer og 8 bytes på 64-bit systemer).

Svar: Patient 1 - Udskrivningsfunktion (call-by-value)

Løsningsfil: `files/solution/patient1_sol.c`

Forventet output:

```
Patient Information:
###
 PatientID: 13
 age:       44 years
 weight:    80.0 kg
 height:    1.80 m
```

PBL-svar: Den færdige funktion med alle parametre:

```c
#include <stdio.h>

void printPatientInfo(int id, int age, float weight, float height) {
    printf("Patient Information:\n");
    printf("###\n");
    printf(" PatientID:\t%d\n", id);
    printf(" age:\t\t%d years\n", age);
    printf(" weight:\t%.1f kg\n", weight);
    printf(" height:\t%.2f m\n", height);
}

int main() {
    int id = 13;
    int age = 44;
    float weight = 80;
```

```
    float height = 1.80;
    printPatientInfo(id, age, weight, height);
    return 0;
}
```

Svar: Patient 2 - Flere patienter med BMI

Løsningsfil: `files/solution/patient2_sol.c`

Forventet output:

```
Patient Information for 4 patients
###
 PatientID: 1
 age:       13 years
 weight:    80.0 kg
 height:    1.80 m
 BMI:       24.7
###
 PatientID: 3
 age:       44 years
 weight:    99.0 kg
 height:    1.62 m
 BMI:       37.7
###
 PatientID: 52
 age:       51 years
 weight:    100.0 kg
 height:    1.30 m
 BMI:       59.2
###
 PatientID: 23
 age:       100 years
 weight:    50.0 kg
 height:    1.74 m
 BMI:       16.5
```

PBL-svar: Tilføj udskrivning af BMI-kategorien:

```
#include <stdio.h>

float calculateBMI(float height, float weight) {
    return weight / (height * height);
}

void printPatientInfo(int id, int age, float weight, float height) {
```

```
    printf("###\n");
    printf(" PatientID:\t%d\n", id);
    printf(" age:\t\t%d years\n", age);
    printf(" weight:\t%.1f kg\n", weight);
    printf(" height:\t%.2f m\n", height);
}

int main() {
    int ids[] = {1, 3, 52, 23};
    int ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
    int number_of_patients = sizeof(ids) / sizeof(ids[0]);

    printf("Patient Information for %d patients\n", number_of_patients);

    for (int i = 0; i < number_of_patients; i++) {
        printPatientInfo(ids[i], ages[i], weights[i], heights[i]);
        float bmi = calculateBMI(heights[i], weights[i]);
        printf(" BMI:\t\t%.1f\n", bmi);

        // Udskriv BMI-kategori
        if (bmi < 18.5) {
            printf(" Category:\tUnderweight\n");
        } else if (bmi < 25.0) {
            printf(" Category:\tNormal weight\n");
        } else if (bmi < 30.0) {
            printf(" Category:\tOverweight\n");
        } else {
            printf(" Category:\tObese\n");
        }
    }
    return 0;
}
```

Svar: Records - Udskrivning med call-by-reference

Løsningsfil: `files/solution/records_display_sol.c`

Forventet output:

```
date: Fri Jul 17 10:23:00 2020
ID: 131
notes: Measurement recorded without issues
measurement: 93.30
```

PBL-svar: Udvidet, så det udskriver flere records:

```c
#include <stdio.h>
#include <time.h>

struct Record {
    int id;
    time_t dato;
    char notes[100];
    float measurement;
};

void print_record(struct Record *r) {
    printf("date: %s", ctime(&r->dato));
    printf("ID: %d\n", r->id);
    printf("notes: %s\n", r->notes);
    printf("measurement: %.2f\n\n", r->measurement);
}

int main() {
    struct Record records[] = {
        {131, 1594974180, "Measurement recorded without issues", 93.3},
        {132, 1595060580, "Second measurement", 95.5},
        {133, 1595146980, "Third measurement", 91.2}
    };

    int num_records = sizeof(records) / sizeof(records[0]);

    for (int i = 0; i < num_records; i++) {
        print_record(&records[i]);
    }

    return 0;
}
```

Svar: Patient 3 - Statistikfunktioner

Løsningsfil: `files/solution/patient3_sol.c`

Forventet output:

```
Patient Descriptives for 4 Patients
Age: mean = 52.00, std = 36.01
Weight: mean = 82.25, std = 23.39
Height: mean = 1.62, std = 0.22
```

(Løsningsfilen udskriver i formatet `Age (Years): 52.00(36.01)`)

PBL-svar: Beregn og udskriv statistikken for alle tre målinger i ét loop:

```
#include <stdio.h>
#include <math.h>

void printStatistics(char* measurement, float mean, float std) {
    printf("%s: mean = %.2f, std = %.2f\n", measurement, mean, std);
}

float mean(float x[], int n) {
    float sum = 0;
    for (int i = 0; i < n; i++) {
        sum += x[i];
    }
    return sum / n;
}

float std(float x[], int n) {
    float m = mean(x, n);
    float sum_sq = 0;
    for (int i = 0; i < n; i++) {
        sum_sq += pow(x[i] - m, 2);
    }
    return sqrt(sum_sq / (n - 1));
}

int main() {
    // ages er float (ikke int), så samme mean()- og std()-funktioner kan bruges til
alle målinger
    float ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
    float *measurements[] = {ages, weights, heights};
    char *names[] = {"Age", "Weight", "Height"};
    int number_of_patients = sizeof(ages) / sizeof(ages[0]);

    printf("Patient Descriptives for %i Patients\n", number_of_patients);

    // Beregn og udskriv statistik for alle tre målinger i ét loop
    for (int i = 0; i < 3; i++) {
        printStatistics(names[i], mean(measurements[i], number_of_patients),
std(measurements[i], number_of_patients));
    }

    return 0;
}
```

Filhenvisninger

Alle løsningsfiler ligger i `files/solution/`:

- Funktioner 1 - Kodens læsbarhed: `functions1_with_comments.c` og `functions1_with_functions.c`
- Funktioner 2 - Funktion til BMI-beregning: `functions2_sol.c`
- Funktioner 3 - Udskriv værdi med pointer: `functions3_sol.c`
- Patient 1 - Udskrivningsfunktion: `patient1_sol.c`
- Patient 2 - Flere patienter med BMI: `patient2_sol.c`
- Records - Udskrivning med call-by-reference: `records_display_sol.c`
- Patient 3 - Statistikfunktioner: `patient3_sol.c`